



Maastricht University

Department of Advanced Computing Science

Algorithmic Design

Week 2:

More Divide and Conquer

David Mestel and **Steven Chaplick**

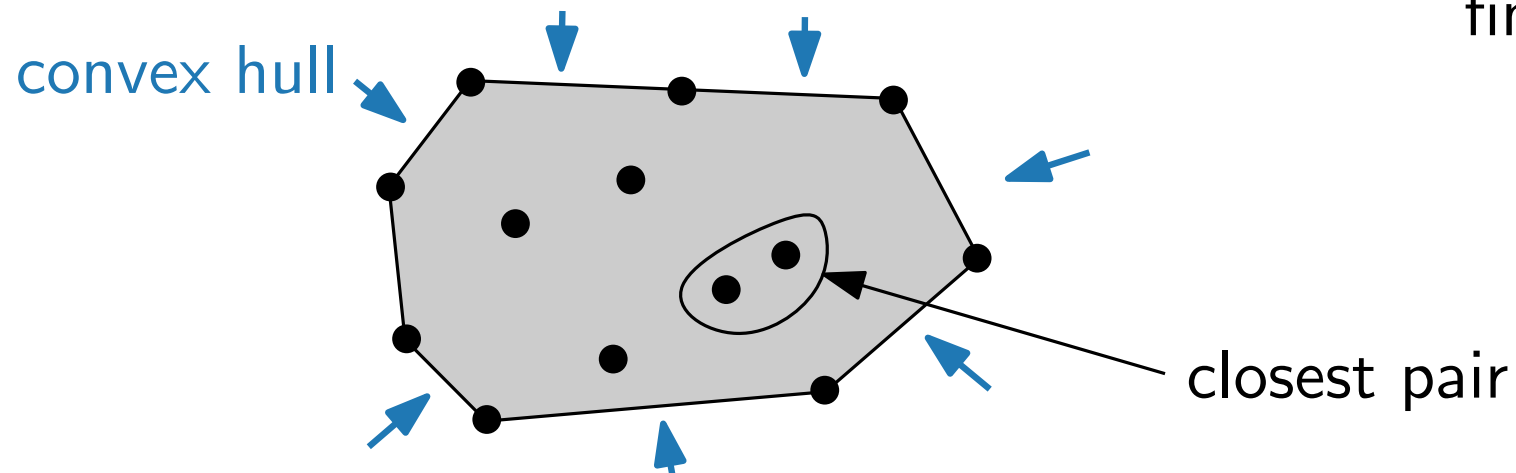
2025-2026 Period 5

Problems to discuss

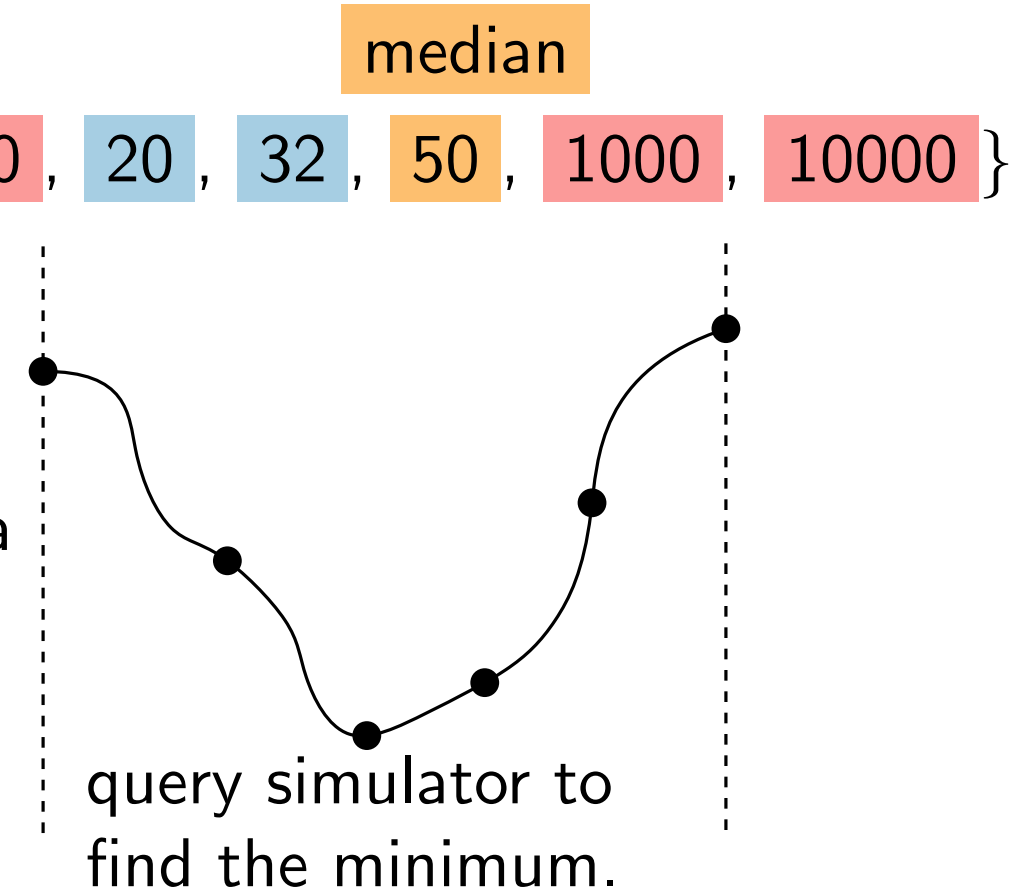
- Selecting the median (or k^{th} smallest) value. { 5, 100, 20, 32, 50, 1000, 10000 }
[GT 9.2] [CLRS 9.3]

- Minimizing a (black-box) convex function.
https://en.wikipedia.org/wiki/Golden-section_search

- Points in $\mathbb{R}^2$. Convex hull and closest pair.



profile of a
simulation



Selection in Unordered sets

median, $k = \lceil \frac{n}{2} \rceil$

Input: A list S of (distinct) n numbers and an index k , with $1 \leq k \leq n$.
 $\{5, 100, 20, 32, 50, 1000, 10000\}$

Output: An k^{th} smallest number in S , i.e., $v \in S$ with exactly $k - 1$ numbers $u \in S$, $u < v$.

Approaches we know:

- Sort the numbers and pick entry k . $O(n \log n)$.
- Slightly faster trick (Week 1 exercise): $O(n \log k)$.

Exercise: Implement split, see PARTITION [CLRS 7.1]

How can we do better? improve this part!

Pivoting (as in quick sort): Pick an element u , and split the list into L, E, R , where $L = \{x : x \in S, x < u\}$, $R = \{y : y \in S, y > u\}$, and $E = \{e : e \in S, e = u\}$.

Idea: index k is in S' , $S' \in \{L, R, E\}$.
 If $S' = E$, done (return u).
 Else, recurse on S' .

Timing? Split $O(n)$

$$T(n) = T(|S'|) + f(n), \\ f(n) \in O(n).$$

Lucky: $|S'| \leq \frac{3}{4}n$ [any cn , $c < 1$]

Master Theorem: $a = 1$,
 $b = \frac{3}{4} < a$, $c = 1$, and $\log_b a < 1$.
 $\Rightarrow T(n) \in O(n^1)$

Unlucky: $|S'| = n - 1$
 $T(n) = T(n - 1) + f(n)$
 $= \sum_{i=0}^{n-1} n - i$
 $\Rightarrow T(n) \in O(n^2)$

Linear Time Selection in Unordered Sets

Input: A *list* S of (distinct) n numbers and an index k , with $1 \leq k \leq n$.

Output: An k^{th} smallest number in S , i.e., $v \in S$ with exactly $k - 1$ numbers $u \in S$, $u < v$.

Algo, use pivoting: Pick an element u , and split the list into L , E , R , where $L = \{x : x \in S, x < u\}$, $R = \{y : y \in S, y > u\}$, and $E = \{e : e \in S, e = u\}$.

Let $S' \in \{L, E, R\}$ with index k . How to pick u ?

Criteria: Guarantee $|S'|$ fraction of n .

Time for splitting?

$$T(n) = T(|S'|) + f(n),$$

$$f(n) \in O(n).$$

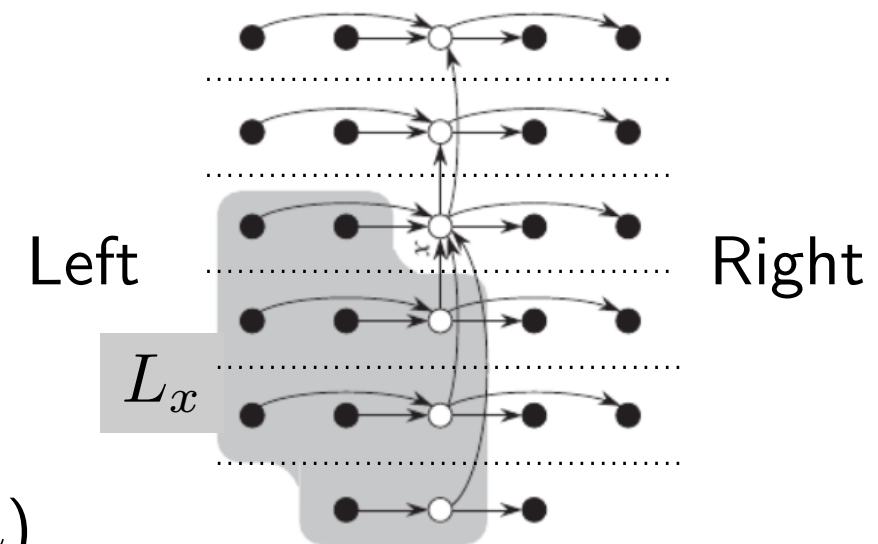
$O(n)$ to split

$\rightsquigarrow O(n)$ to **pick** u

Thought Process:

For small sets, say 5 elements, find median in constant time.

- Break S into many small sets. $O(n)$
- Find median in each set, $O(1)$ per set.
- Many medians... $|M| = \frac{n}{5}$
- x median of medians.
 $|L_x| \geq \frac{1}{5}n$



medians M

Use linear-time selection on M to find x



Linear Time SELECT

SELECT(S, k)

1. Split n elem. into $\lfloor n/5 \rfloor$ sets of 5, and one with $(n \bmod 5)$.
2. Sort each $\lfloor n/5 \rfloor$ sets, and find medians $\rightsquigarrow$ median set M .
3. *Recursively* find the median x of the set M of medians.

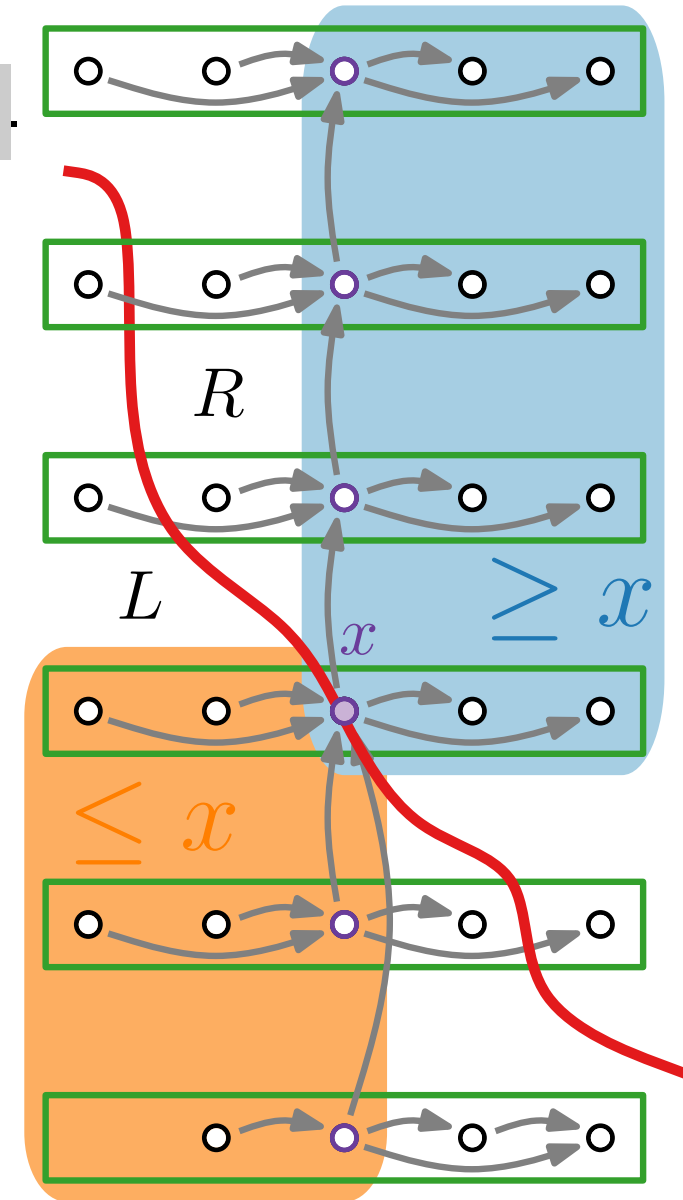
$$x := \text{SELECT}(M, \lceil \frac{|M|}{2} \rceil)$$

4. $(L, R, E) = \text{SPLIT}(S, x)$

5. **if** $k < |L|$ **then**
 | **return** SELECT(L, k)
if $k > |L| + |E|$ **then**
 | **return** SELECT($R, k - |L| - |E|$)
return x

$$|E| + |L| \geq \text{Count}(\bullet) \geq 3 \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 1 \right) \geq \frac{3n}{10} - 3$$

small!



Linear Time SELECT

SELECT(S, k)

1. Split n elem. into $\lfloor n/5 \rfloor$ sets of 5, and one with $(n \bmod 5)$.
2. Sort each $\lfloor n/5 \rfloor$ sets, and find medians $\rightsquigarrow$ median set M .
3. *Recursively* find the median x of the set M of medians.

$$x := \text{SELECT}(M, \lceil \frac{|M|}{2} \rceil)$$

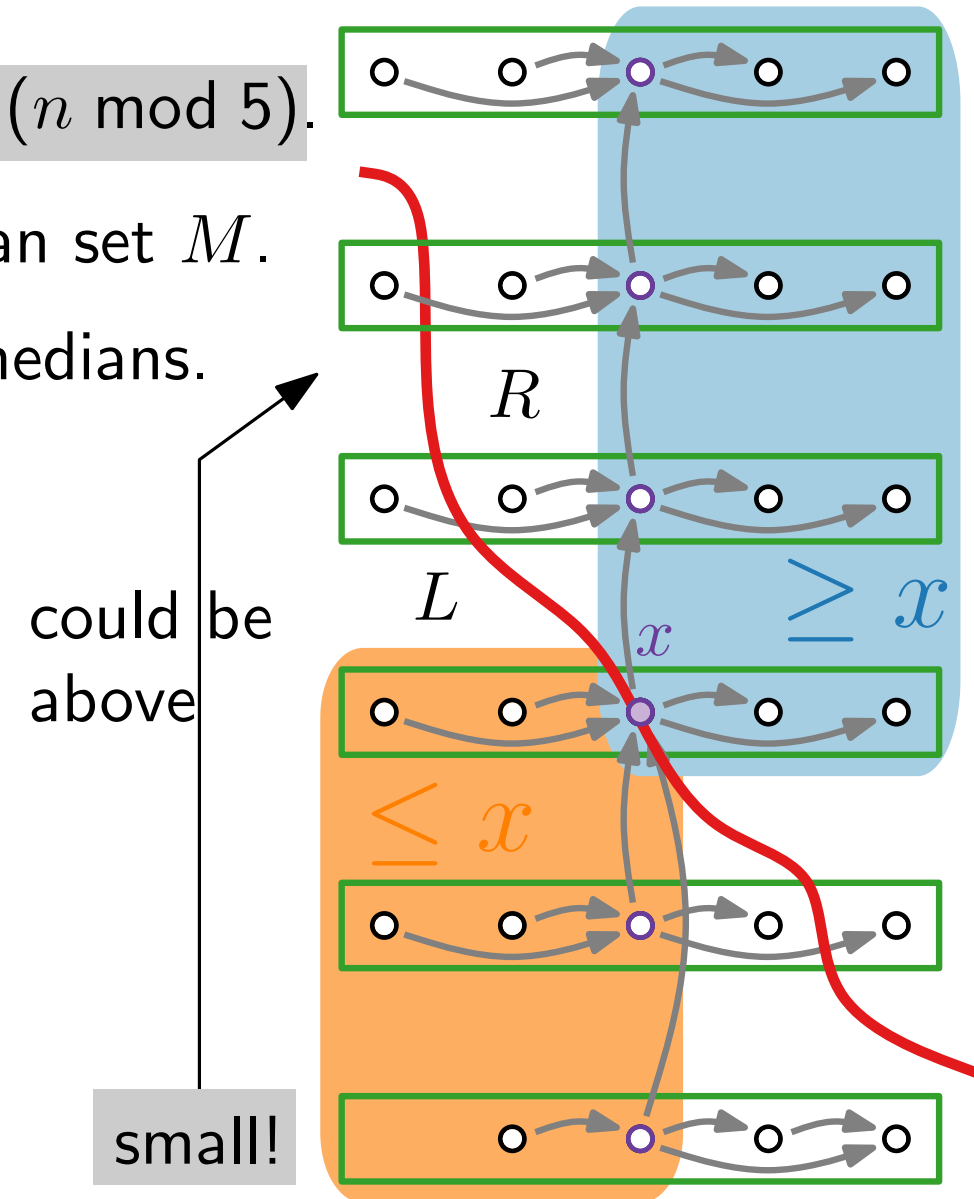
4. $(L, R, E) = \text{SPLIT}(S, x)$

5. **if** $k < |L|$ **then**
 return SELECT(L, k)
if $k > |L| + |E|$ **then**
 return SELECT($R, k - |L| - |E|$)
return x

$\leq 7n/10 + 3$ elements

$$|E| + |L| \geq \text{Count}(\bullet) \geq 3 \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 1 \right) \geq \frac{3n}{10} - 3$$

$$|E| + |R| \geq \text{Count}(\bullet) \geq 3 \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 1 \right) \geq \frac{3n}{10} - 3$$



Linear Time SELECT: Runtime Analysis

Recall. SPLIT(S, x) and finding medians in the small sets, both linear $O(n)$, say bn steps total.

What is the recurrence for SELECT?

Analysis.
$$T(n) \leq \begin{cases} \overbrace{T(\lceil n/5 \rceil) + T(7n/10 + 3) + bn}^{\text{line 3}} & \text{for } n \geq n_0, \\ \mathcal{O}(1) & \text{otherwise (base case).} \end{cases}$$
 line 5

Need to Show. There are $c, n_0 > 0$, such that for all $n \geq n_0$: $T(n) \leq cn$.

To find c , suppose we know c when n is small.

Then: $T(n) \leq c \cdot (n/5 + 1) + c \cdot (7n/10 + 3) + bn$

$$\begin{aligned} &= c \cdot (9n/10 + 4) + bn &= cn - (c \cdot (\frac{n}{10} - 4) - bn) \\ & &= cn \left(1 - \left(\frac{1}{10} - \frac{4}{n} - \frac{b}{c} \right) \right) \end{aligned}$$

?! ≥ 0

try: $\frac{1}{20} \leq \frac{4}{n}$ and $\frac{1}{20} \leq \frac{b}{c}$. $\Leftrightarrow n \geq 80$, and $c \geq 20b$.

Thus, try $n_0 \geq 80$ and $c \geq 20b$.

Linear Time SELECT: Runtime Analysis (proof)

Theorem: For any n element set S , the runtime of $\text{SELECT}(S, k)$ is $O(n)$.

Use b as in prev. slide for SPLIT and finding medians.

Let α be the steps needed when $n \leq 80$. (base case).

For example, by sorting and picking the k^{th} element.

Pick $n_0 = \max\{80, \alpha\}$, $c = \max\{20b, \alpha\}$.

Use induction.

Base Case: When $1 \leq n \leq n_0$, use at most α steps $\leq \alpha n \leq cn$. Done.

Ind. Case: Suppose $T(n) \leq cn$ for all $n < n'$. Show it works for n'

$$\begin{aligned} T(n') &\leq T(\lceil n'/5 \rceil) + T(7n'/10 + 3) + bn' \\ &= c \cdot (9n'/10 + 4) + bn' \leq cn' \quad (\text{as in prev. slide}) \end{aligned}$$

Done. :)

Linear Time Selection in Unordered Sets

Input: A *list* S of (distinct) n numbers and an index k , with $1 \leq k \leq n$.

Output: An k^{th} smallest number in S , i.e., $v \in S$ with exactly $k - 1$ numbers $u \in S$, $u < v$.

Theorem: For any n element set S , the runtime of $\text{SELECT}(S, k)$ is $O(n)$.

Ideas of SELECT .

- Split in small sets (5 elements). Find the median in each one.
- Find the median m of the medians recursively.
- Use m as a **pivot** to divide (prune) S , and recurse.

References: [GT 9.2] [CLRS 9.3]

Exercise: Implement $\text{SPLIT}(S, x)$ in-place, i.e., by only swapping elements inside the array S .

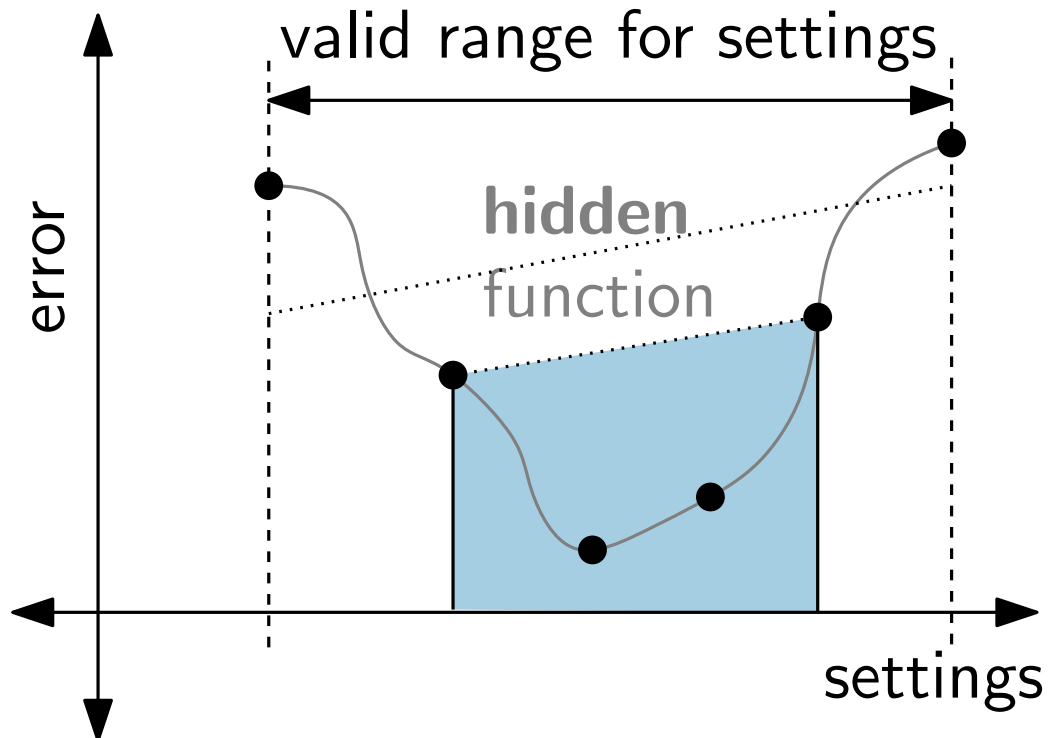
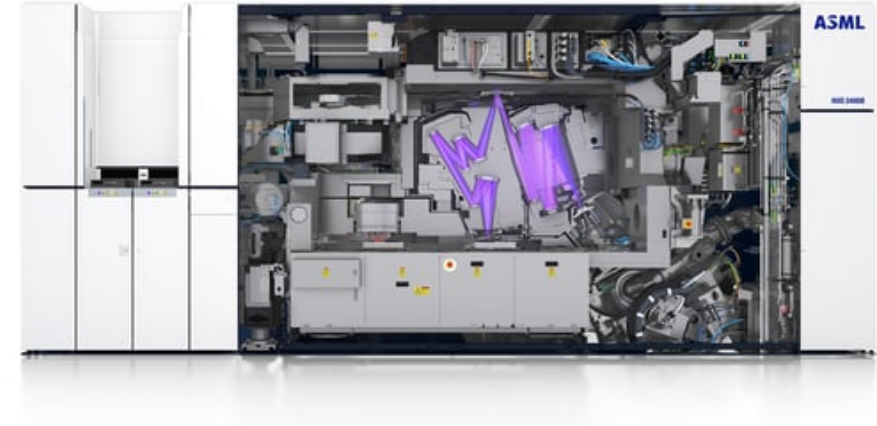
Black-box Minimization

Digital Twin: computational simulation of a physical system.
eg, can be used to find “ideal” settings

Problem: Running a simulation is **computationally expensive**.

Minimize the number of calls to the simulator.

What is a simulation? Given certain settings x for the machine,
calculate the error (quality) y in the result.



For which values x should we run the simulator?

Special Setting: One minimum in the valid range.

Geometric viewpoint. **Convex**: any line pierces our function at most twice.

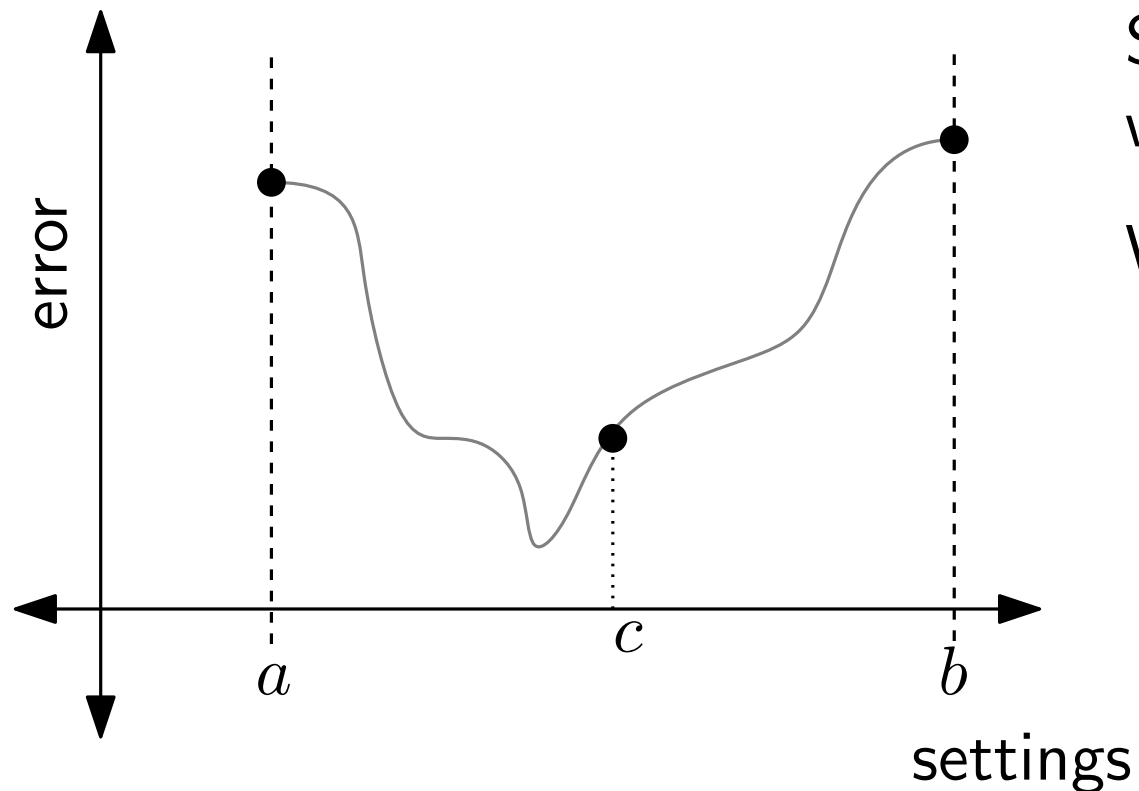
Black-box Minimization of a Convex Function

Input: Given a convex function $f : [a, b] \rightarrow \mathbb{R}$, and a threshold $t > 0$.

(i.e., f takes in x in the real interval $[a, b]$ ($a < b$), and returns back a real number.)

Output: $x^* \in [a, b]$ such that for any x , if $x < x^* - t$ or $x > x^* + t$, then $f(x) > f(x^*)$.

Goal: Determine an algorithm to find x^* efficiently.



How can we **search** the continuous space $[a, b]$?

Start with $f(a), f(b)$. Try also midpoint: c
wlog $f(a) < f(b)$

Where can we find x^* ?

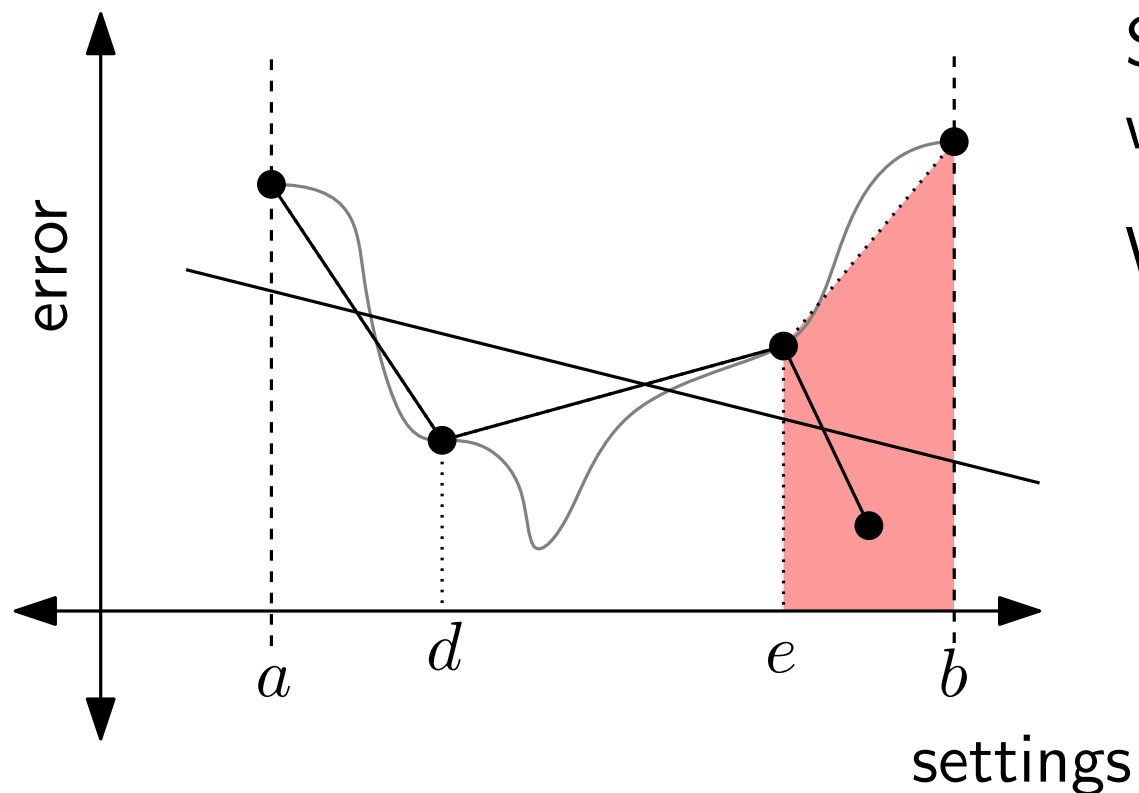
Black-box Minimization of a Convex Function

Input: Given a convex function $f : [a, b] \rightarrow \mathbb{R}$, and a threshold $t > 0$.

(i.e., f takes in x in the real interval $[a, b]$ ($a < b$), and returns back a real number.)

Output: $x^* \in [a, b]$ such that for any x , if $x < x^* - t$ or $x > x^* + t$, then $f(x) > f(x^*)$.

Goal: Determine an algorithm to find x^* efficiently.



How can we **search** the continuous space $[a, b]$?

Start with $f(a), f(b)$. Try two “midpoints” d, e
 wlog $f(a) < f(b)$ Suppose $f(a) > f(e) > f(d)$

Where can we find x^* ?

Can x^* be in the red region?

No, otherwise break convexity!

(use intermediate value theorem from Calculus)

Black-box Minimization of a Convex Function: Algorithm

Input: Given a convex function $f : [a, b] \rightarrow \mathbb{R}$, and a threshold $t > 0$.

(i.e., f takes in x in the real interval $[a, b]$ ($a < b$), and returns back a real number.)

Output: $x^* \in [a, b]$ such that for any x , if $x < x^* - t$ or $x > x^* + t$, then $f(x) > f(x^*)$.

Goal: Determine an algorithm to find x^* efficiently.

Golden Ratio:

$$\varphi = 1 + r$$

Exercise: write out pseudocode for this schematic

Use recursion! What is the base case?

But how to select the spacing between these values?

Idea: ensure always discard “big enough” fraction.

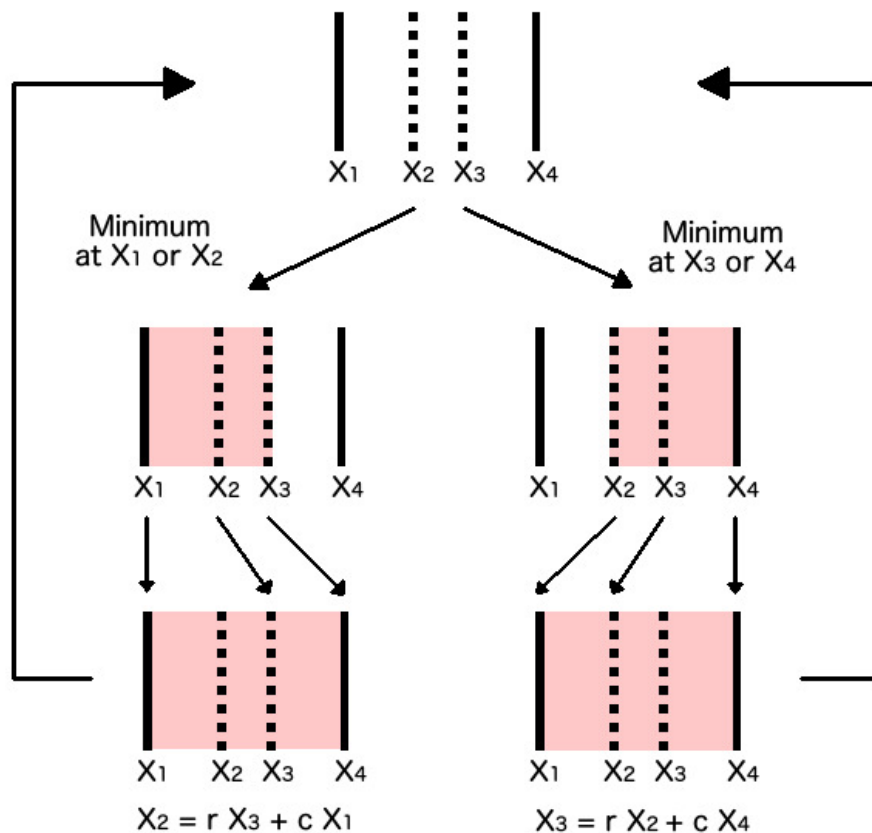
easier: always discard the same fraction

Relative interval lengths: 1, r , 1. ($r \in [0, 1]$)

Next iteration: r , $(1 - r)$, r .

Thus, $\frac{1}{r} = \frac{r}{1-r} \Leftrightarrow r^2 + r - 1 = 0. \Leftrightarrow r = \frac{-1+\sqrt{5}}{2}$

Each iteration the interval scales by $\frac{1+r}{2+r} \leq 0.62$



Black-box Minimization of a Convex Function: Summary

Input: Given a convex function $f : [a, b] \rightarrow \mathbb{R}$, and a threshold $t > 0$.

(i.e., f takes in x in the real interval $[a, b]$ ($a < b$), and returns back a real number.)

Output: $x^* \in [a, b]$ such that for any x , if $x < x^* - t$ or $x > x^* + t$, then $f(x) > f(x^*)$.

Goal: Determine an algorithm to find x^* efficiently.

Using the 4-point idea, repeatedly divide (prune) the search the interval $[a, b]$ to find x^* .

Similar to a binary search, each pruning step removes a fraction of the interval.

Here at least 0.28 of the interval is removed.

Thus, after $O(\log(\frac{a-b}{t}))$ repetitions, such an x^* can be found.

Exercise: write out pseudocode for this.

More details: https://en.wikipedia.org/wiki/Golden-section_search

Convex Hulls, motivation: mixing things

Given: (liquid) substances $s_1, s_2, s_3, \dots$ made up of several different components $A, B, C, \dots$. The proportions of these components vary between the substances.

Question: Can we mix new substances $q_1, q_2, \dots$ with certain target proportions?

Example:

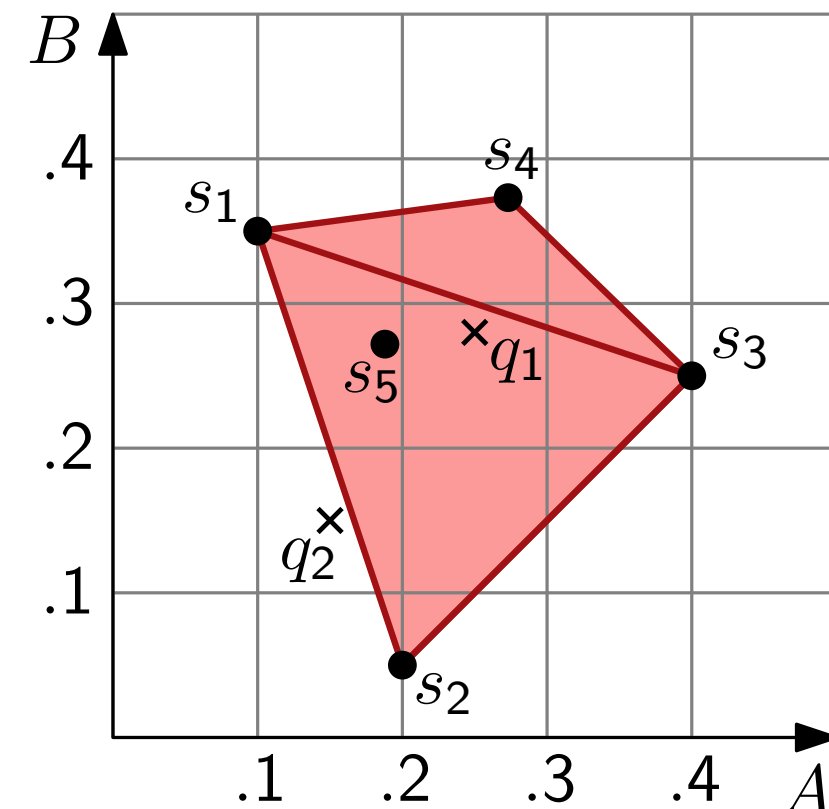
subst.	comp. A	comp. B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

can we mix

q_1	25 %	28 %
q_2	15 %	15 %

using s_1, s_2, s_3 ?

yes!
no!



The region of substances mixable with a set $S \subset \mathbb{R}^d$ of substances corresponds to the so-called **convex hull** $\text{CH}(S)$ of S .

Convex hull: definitions, characterizations, analogies

A physical analogy for the convex hull in $\mathbb{R}^2$:

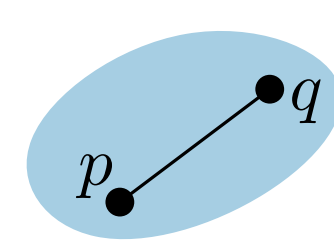
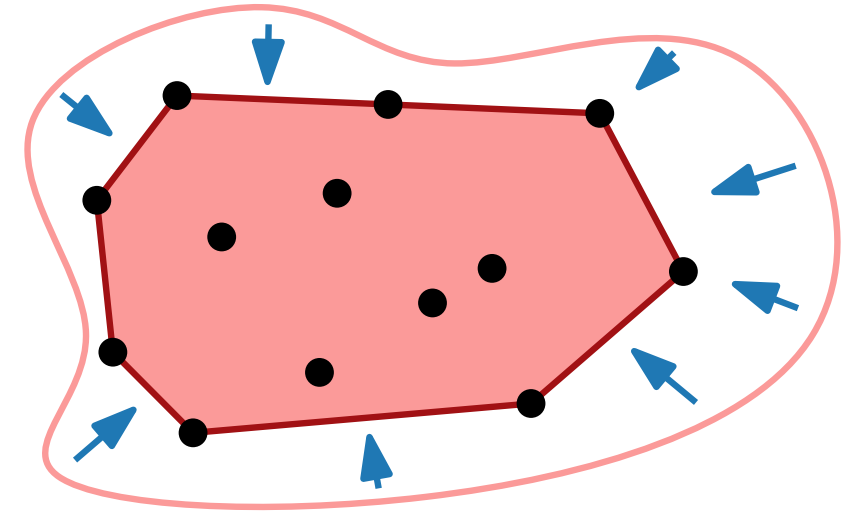
- place a nail for each point in the plane
- place a stretched rubber band around all nails
- let go
- convex hull = area enclosed by the rubber band

A set $S \subseteq \mathbb{R}^d$ is called **convex** if for each pair of points $p, q \in S$, the line segment $\overline{pq}$ is contained in S .

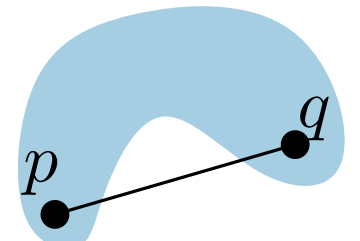
The convex hull $\text{CH}(S)$ of $S \subseteq \mathbb{R}^d$ is the intersection of all convex subsets of $\mathbb{R}^d$ containing S .

Lemma 1. The convex hull of a *finite point set* S in $\mathbb{R}^2$ is the unique convex polygon whose corners are points in S and that contains all points of S .

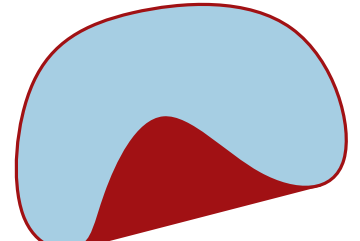
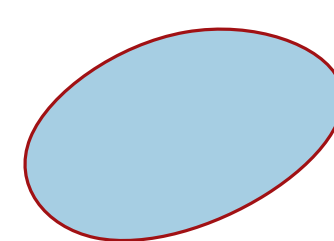
(without proof)



convex



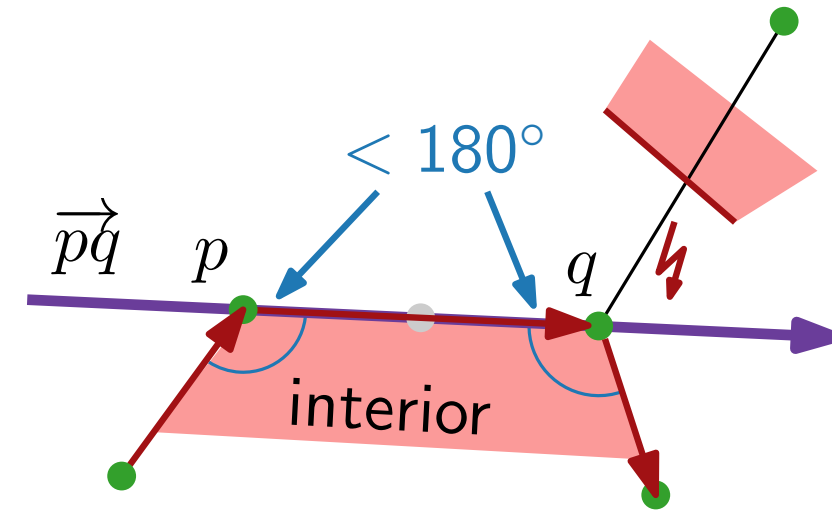
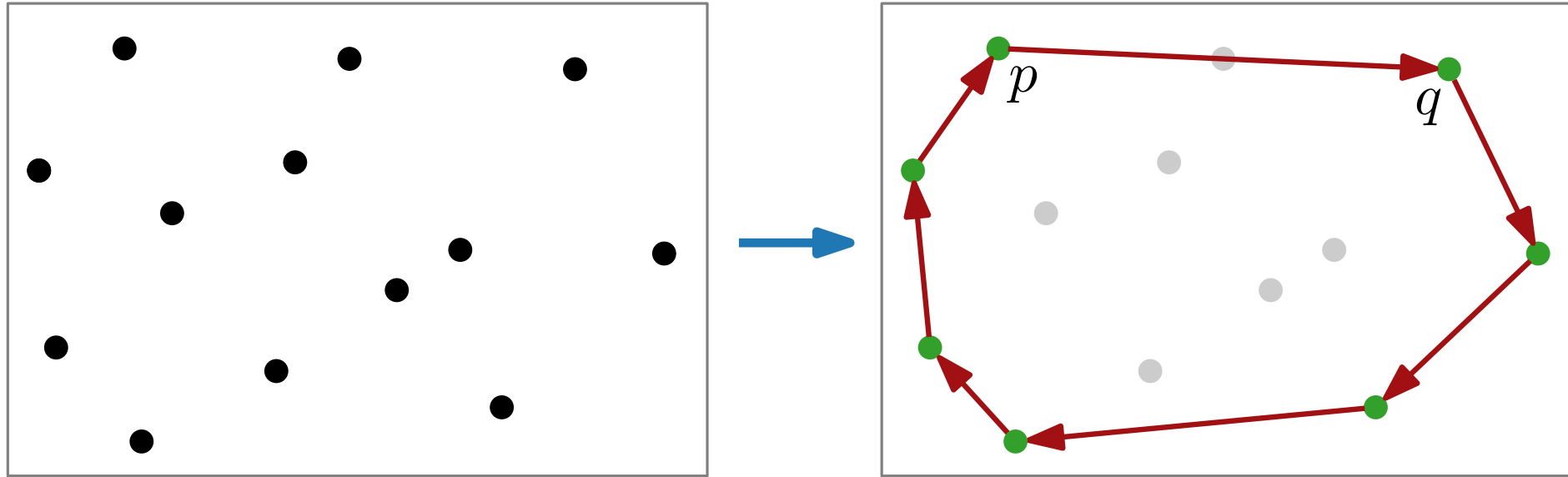
not convex



Algorithmic framework & first algorithmic idea

Input: set S of n points in the plane $\mathbb{R}^2$

Output: list of corners of $\text{CH}(S)$ in clockwise order



Idea: Test for each ordered pair $(p, q) \in S \times S$ if it belongs to the convex hull.

Observation. (p, q) is an edge of $\text{CH}(S) \iff$

each point in S lies

- strictly to the right of the directed line $\vec{pq}$ or
- on the line segment $\overline{pq}$

A first algorithm

Important: this
orientation test takes $O(1)$ time!

FirstConvexHull(S)

$E \leftarrow \emptyset$

foreach $(p, q) \in S \times S$ with $p \neq q$ **do** // test: (p, q) edge of $\text{CH}(S)$?

$valid \leftarrow true$

foreach $r \in S$ **do**

if not (r strictly right of $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

$valid \leftarrow false$

if $valid$ **then**

$E \leftarrow E \cup \{(p, q)\}$

from E construct sorted list L of corners of $\text{CH}(S)$

return L

r strictly right of $\overrightarrow{pq}$



$$\begin{vmatrix} x_p & x_q & x_r \\ y_p & y_q & y_r \\ 1 & 1 & 1 \end{vmatrix} > 0$$

r strictly left of $\overrightarrow{pq}$

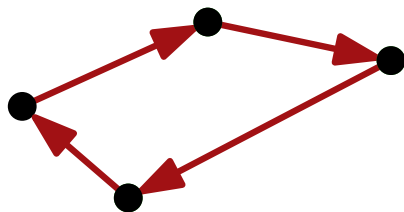


$$\begin{vmatrix} x_p & x_q & x_r \\ y_p & y_q & y_r \\ 1 & 1 & 1 \end{vmatrix} < 0$$

p, q, r collinear



$$\begin{vmatrix} x_p & x_q & x_r \\ y_p & y_q & y_r \\ 1 & 1 & 1 \end{vmatrix} = 0$$



$\left(\det(a, b, c) \text{ equals the signed volume of the parallelepiped spanned by column vectors } a, b, c \right)$

Running time analysis

FirstConvexHull(S)

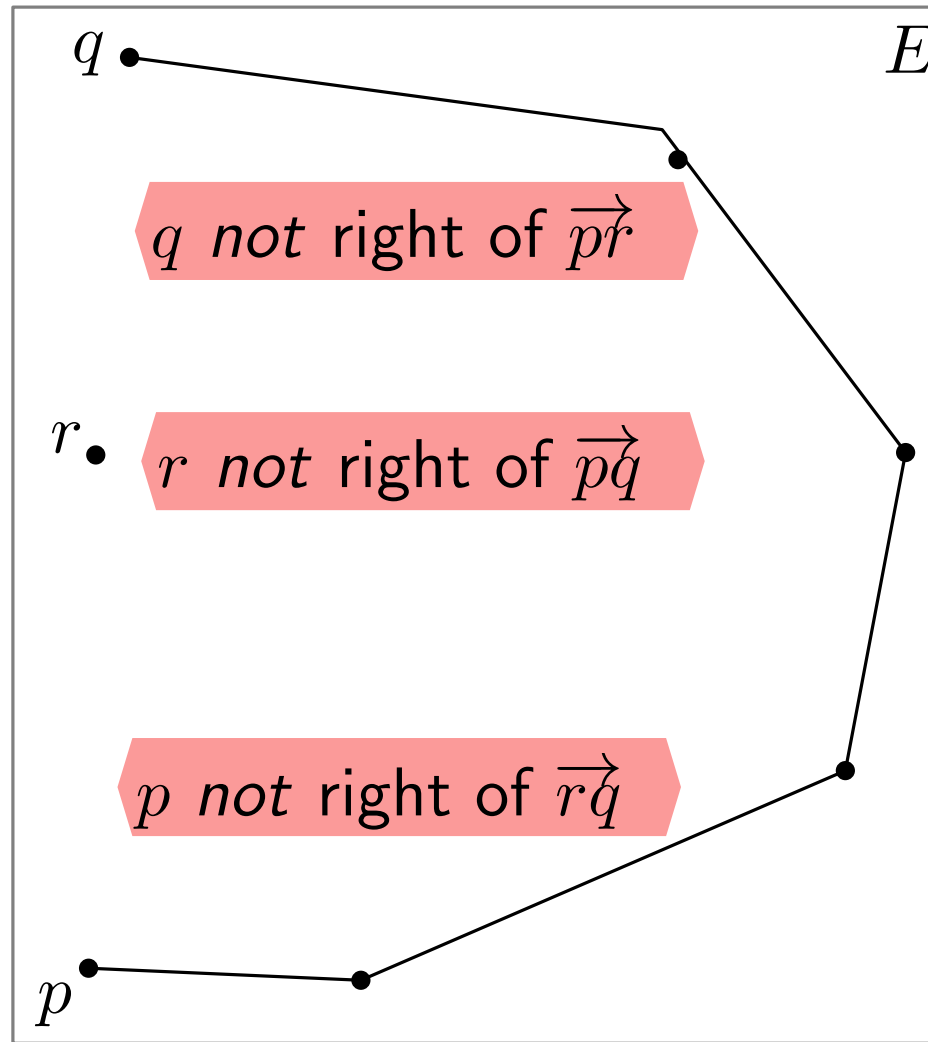
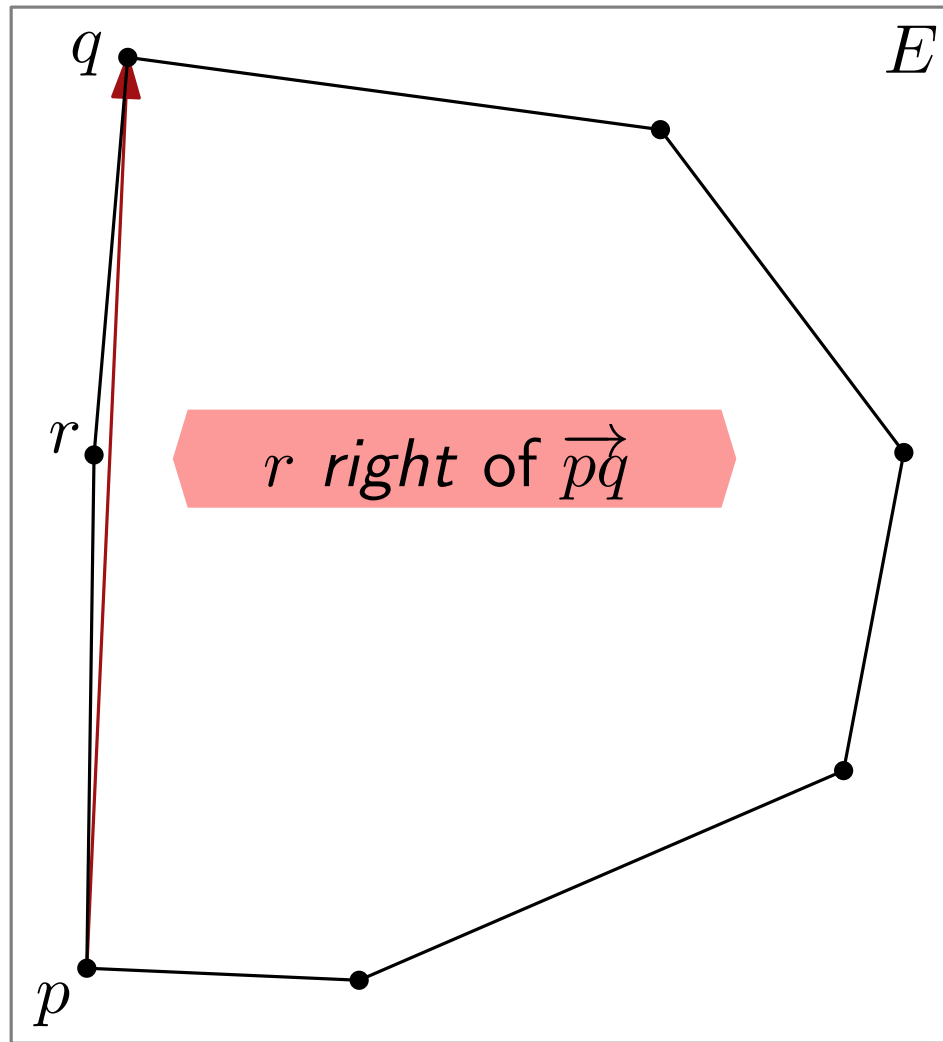
$E \leftarrow \emptyset$

foreach $(p, q) \in S \times S$ with $p \neq q$ do	$(n^2 - n) \cdot$	
$valid \leftarrow true$		
foreach $r \in S$ do	$n \cdot$	
if not (r strictly right of $\overrightarrow{pq}$ or $r \in \overline{pq}$) then	$\Theta(1)$	$\Theta(n)$
$valid \leftarrow false$		
if $valid$ then		
$E \leftarrow E \cup \{(p, q)\}$		
from E construct sorted list L of vertices of $CH(S)$		$\Theta(n^3)$
return L		$O(n^2)$

Lemma. We can compute the convex hull of n points in the plane in $\Theta(n^3)$ time.

Robustness

if not (r strictly right of $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
 $\quad \text{valid} \leftarrow \text{false}$



The orientation test involves only additions and multiplications $\rightarrow$ no floating point errors if the coordinates are integer!

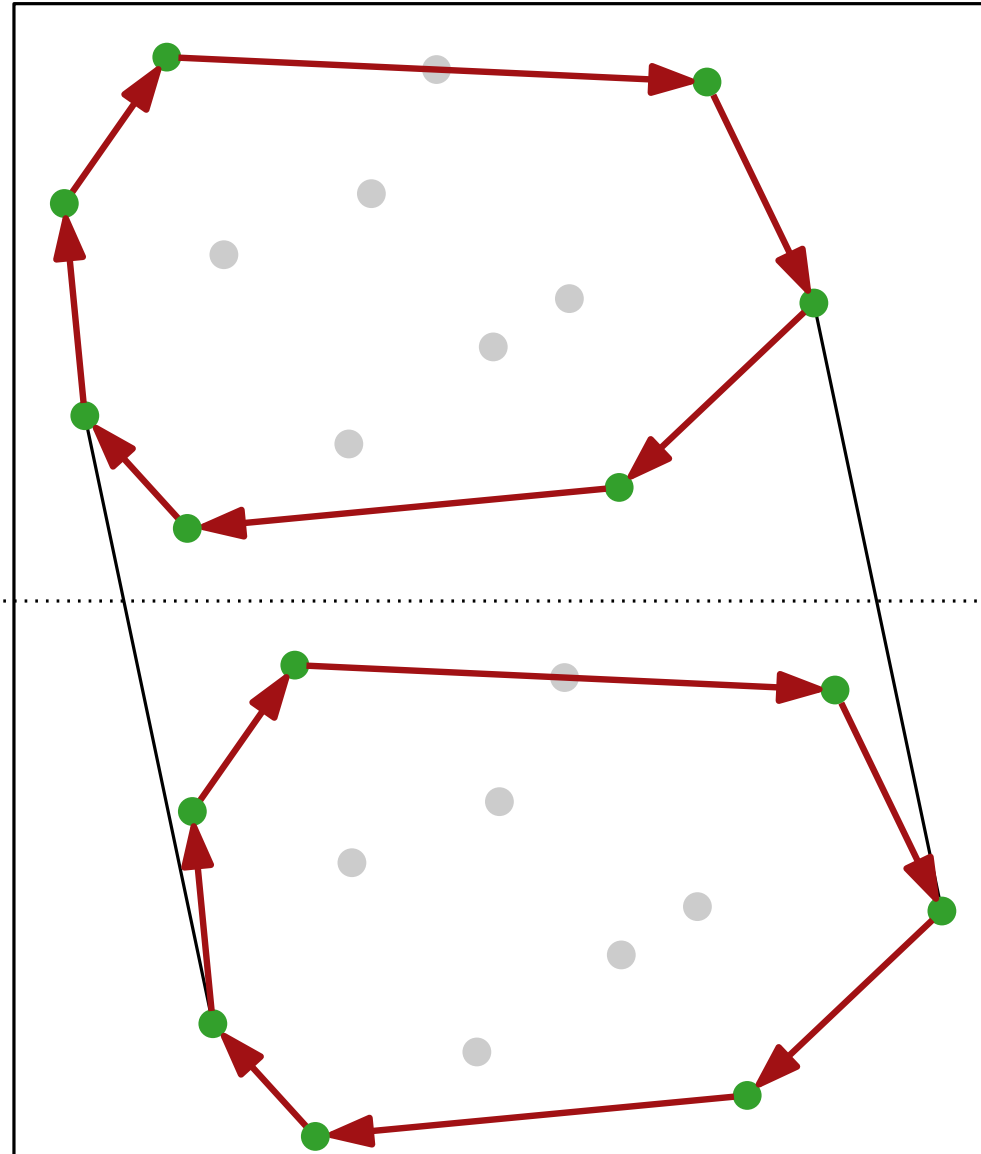
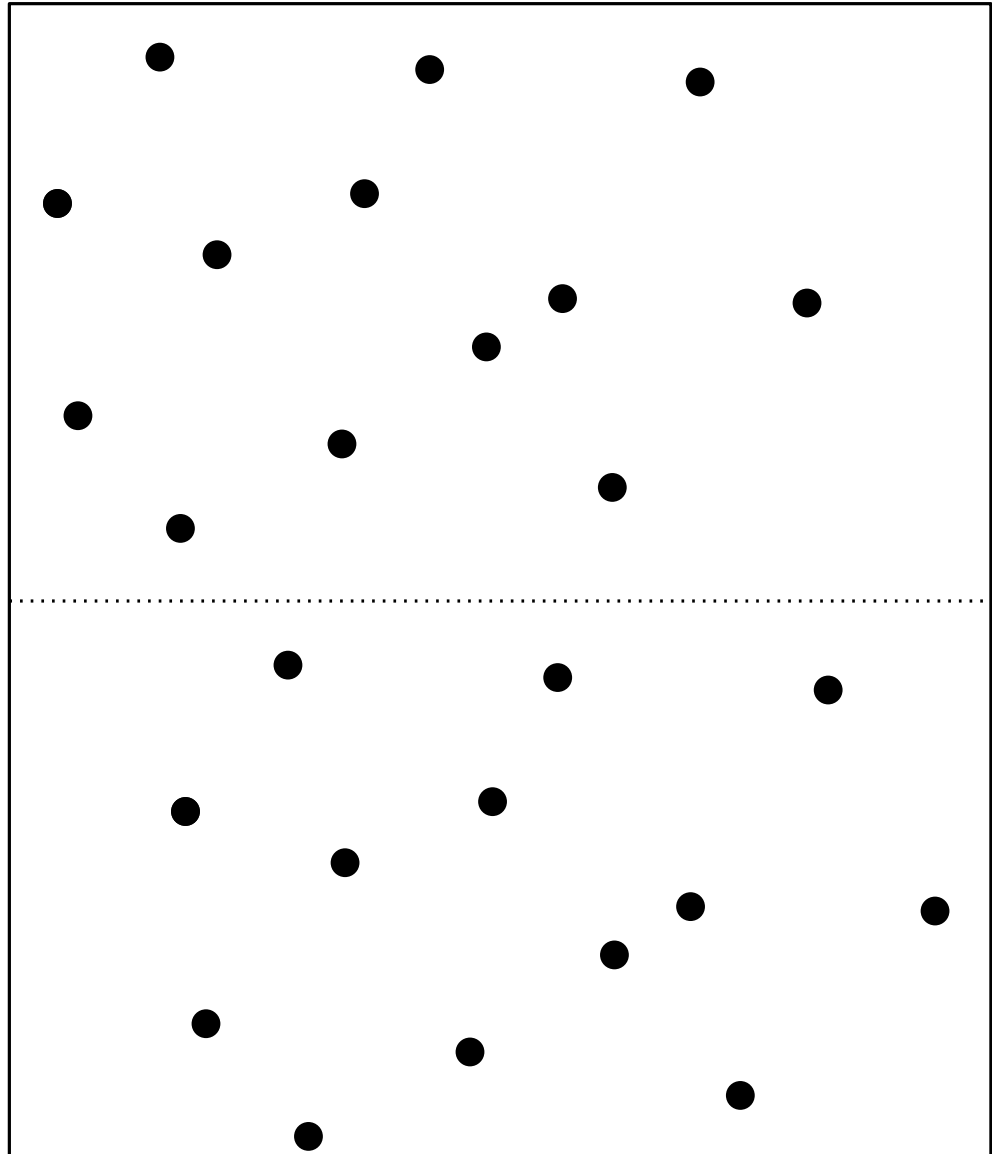
When working with floating point coordinates, the orientation test might return the wrong answer :-)

Algorithm FirstConvexHull is not **robust** under rounding errors: it could crash (depending on the implementation); or return an output that is not a polygon.

But that wasn't Divide and Conquer!

Ok, but now input/output is clear, so let's try dividing.

Sort by y -coordinate. Split as in merge sort.



How to conquer (merge) two convex hulls?

Just need to find these two connections!

Convex Hulls: Find the bridge

Sort by y -coordinate. Split as in merge sort.

How to conquer
(merge) two
convex hulls?

Just need to
find these two
connections!

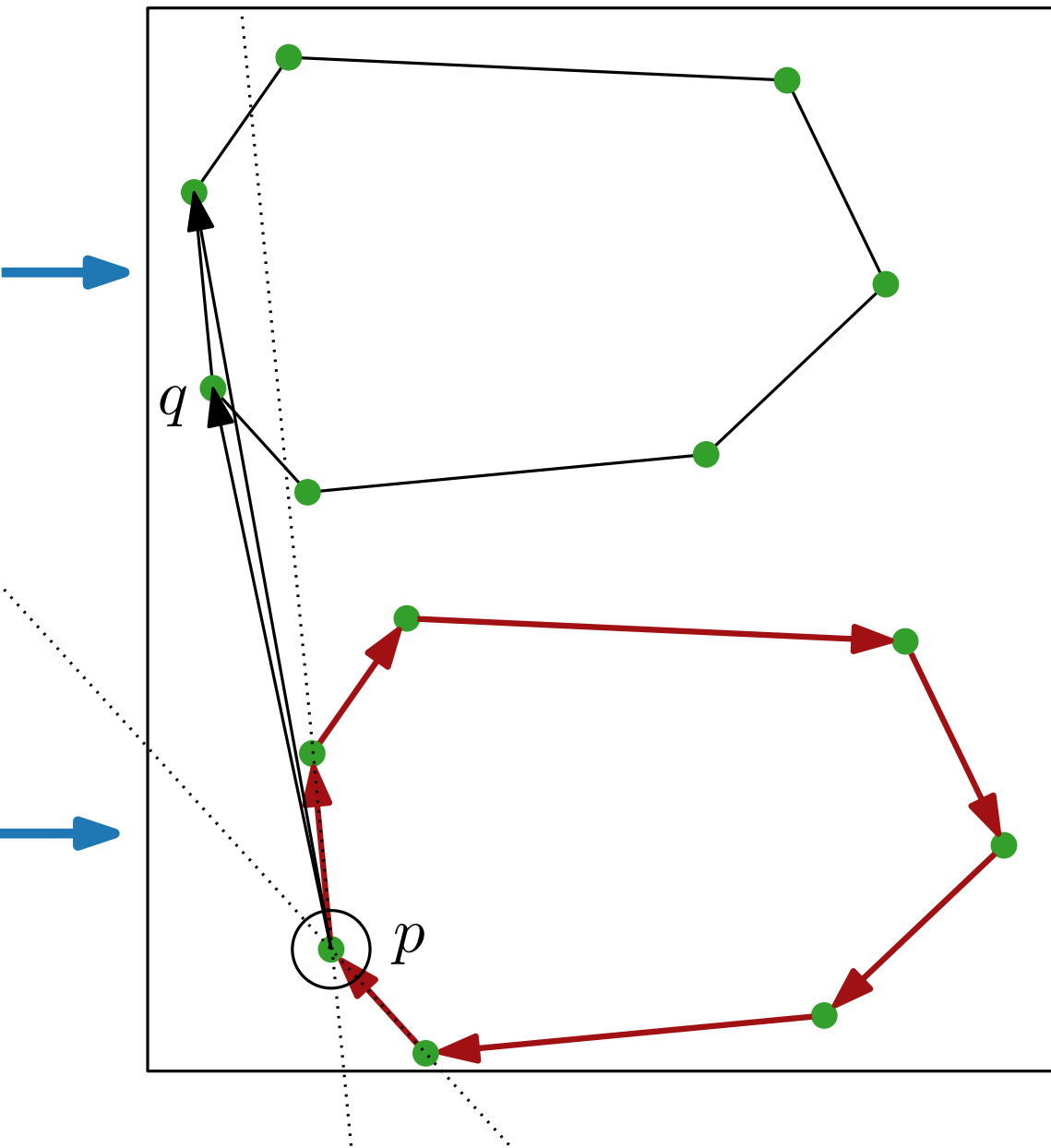
Start from the minimum x -coordinate of the lower hull.
and the minimum x -coordinate of the upper hull.

→ Compare with the two nearby edges in the lower Hull.
If q is left of both, revise p to be the point
clock-wise earlier. (and repeat)]

If q is between the two nearby edges,

→ consider q' clock-wise earlier than q .
If q' is left of $\overrightarrow{pq}$, revise q to be q' . (repeat)]

Let's formalize these ideas!



Convex Hulls: Find the bridge

Given two convex hulls H_1 , H_2 : as clockwise ordered point sets, H_1 is strictly above H_2 .
Merge H_1 and H_2 to make a convex hull H of both.

Let $q = (x_q, y_q)$ be point with minimum x -coordinate in H_1

Let $p = (x_p, y_p)$ be point with minimum x -coordinate in H_2

If $x_q \leq x_p$ (as in the previous slide).

$O(n)$ Trace H_2 counter-clockwise starting from p (revising p in each step) until q is between the two lines of p .

Claim: The convex hull H_2 is to the right of line segment $\overrightarrow{pq}$.

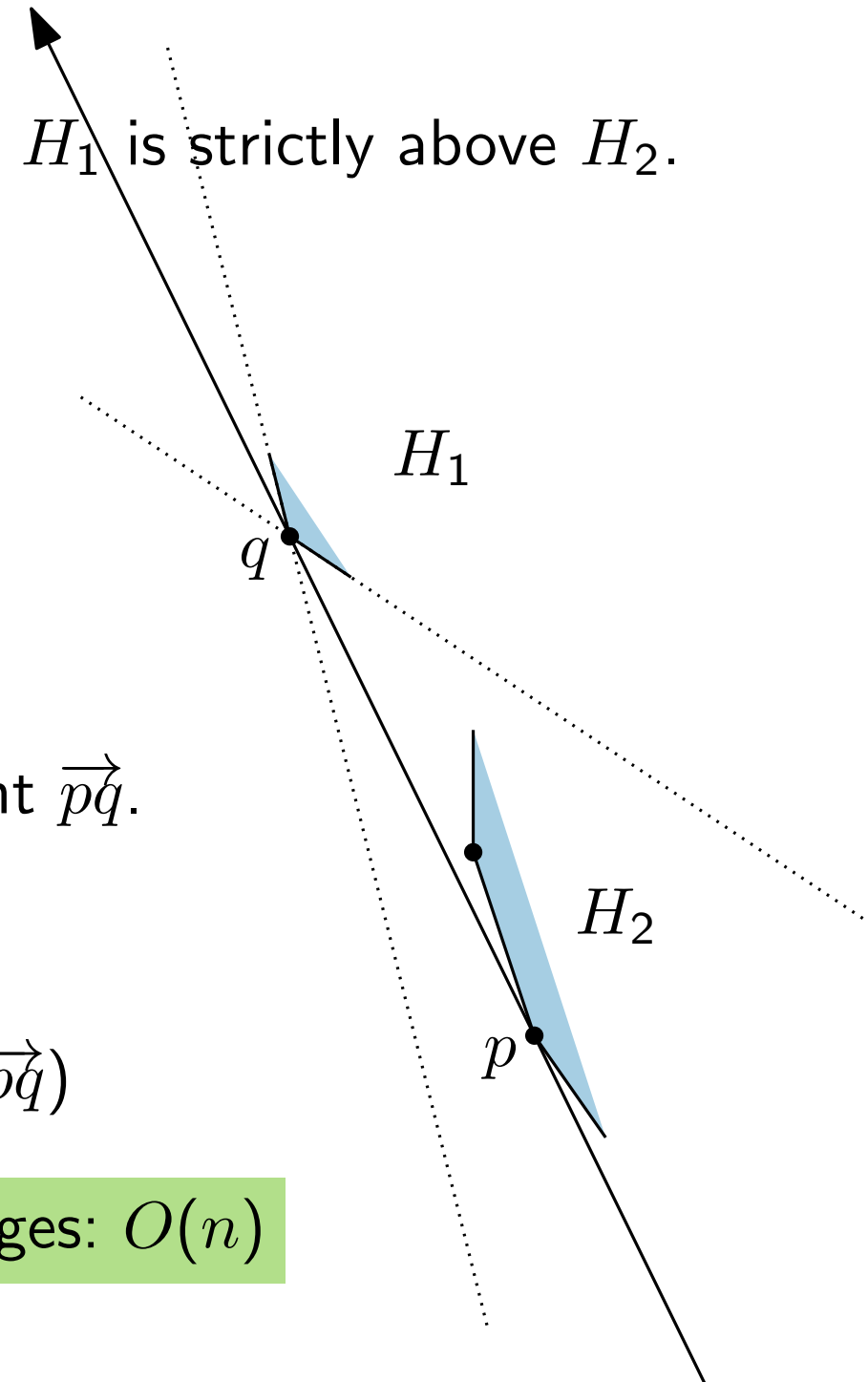
$O(n)$ Trace H_1 counter-clockwise from q (revising q in each step) until p is between the two lines of q .

Claim: H_1 is to the right of $\overrightarrow{pq}$ (and H_2 is still right of $\overrightarrow{pq}$)

Thus, $\overrightarrow{pq}$ is on H

Else (similar, exercise: work it out!) $O(n)$

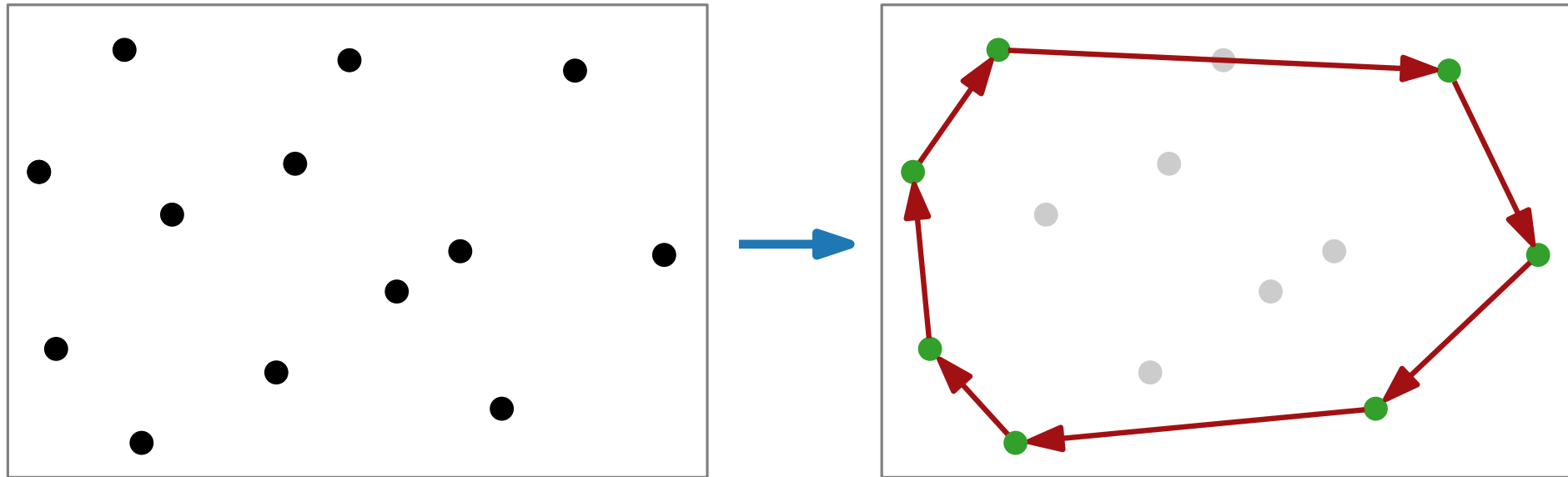
Finding both bridges: $O(n)$



Convex Hulls: Summary

Input: set S of n points in the plane $\mathbb{R}^2$

Output: list of corners of $\text{CH}(S)$ in clockwise order



Sort points by y -coordinate. $O(n \log n)$

Recursively compute convex hull of top, and bottom half: $2T(n/2)$.

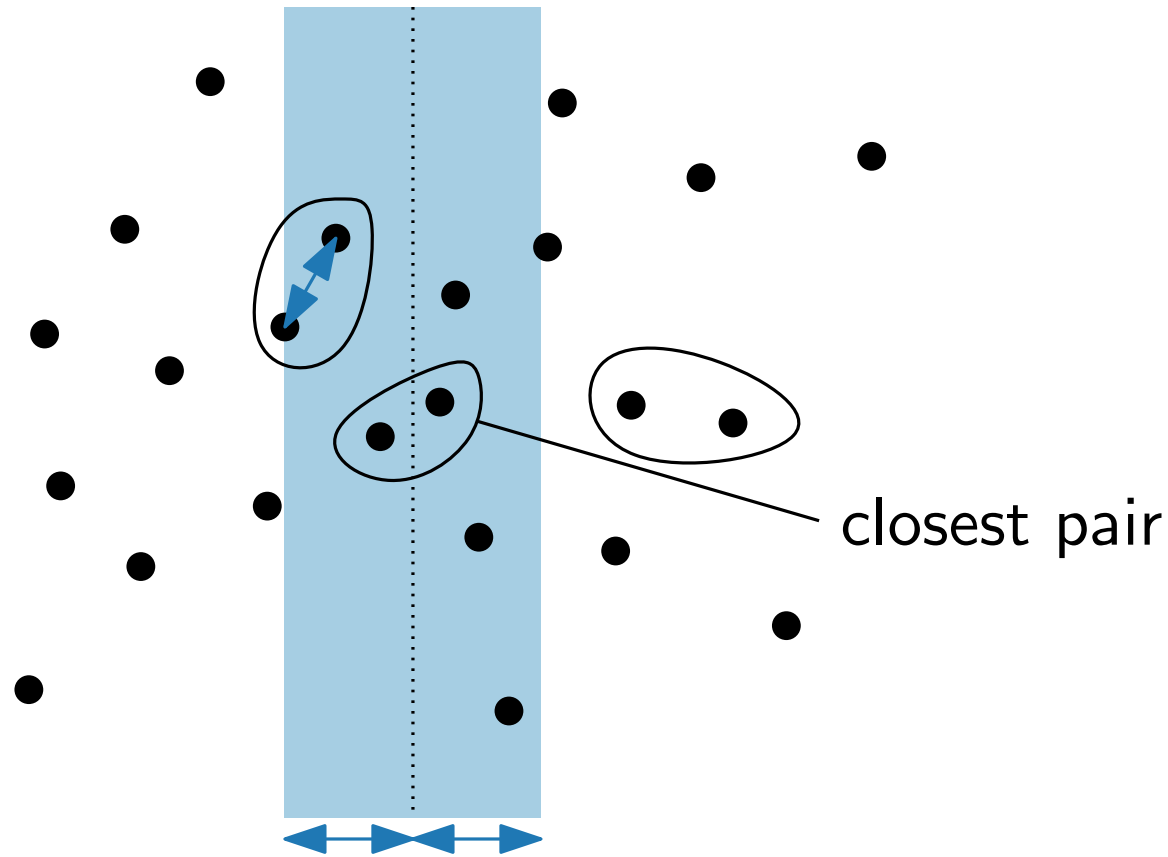
Merge by building bridges $O(n)$. $T(n) = 2 \cdot T(n/2) + f(n)$ where $f(n) \in O(n)$.

Total: $O(n \log n + T(n)) = O(n \log n)$

Closest Pair of Points

Input: A set of points in the plane, $\{p_1 = (x_1, y_1), p_2 = (x_2, y_2), \dots, p_n = (x_n, y_n)\}$

Output: The closest pair of points: that is, the pair $p_i \neq p_j$ for which the distance between p_i and p_j , that is, $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$, is minimized.



Closest Pair of Points

Input: A set of points in the plane, $\{p_1 = (x_1, y_1), p_2 = (x_2, y_2), \dots, p_n = (x_n, y_n)\}$

Output: The closest pair of points: that is, the pair $p_i \neq p_j$ for which the distance between p_i and p_j , that is, $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$, is minimized.

Overview of the algorithm:

- Find a value x for which exactly half the points have $x_i < x$, and half have $x_i > x$. Split the points into two groups, L and R .
- Recursively find the closest pair in L and in R . These pairs are $p_L, q_L \in L$ and $p_R, q_R \in R$, with distances d_L and d_R respectively. Let $d = \min\{d_L, d_R\}$.
- Check if there is a point in L and a point in R closer than d apart from each other. First, discard all points with $x_i < x - d$ or $x_i > x + d$. Second, sort the remaining points by y-coordinate.
- Go through this sorted list, and for each point, compute its distance to the **seven** subsequent points in the list. Let p_M, q_M be the closest pair found in this way.

Closest Pair of Points : Analysis

Exercise:

1. Lemma for correctness. Prove: any square of size $d \times d$ in the plane contains at most four points of L .
2. (correctness) Prove: The algorithm finds the closest pair. (hint: essential case is when the closest pair is split between L and R).
3. Write down the pseudocode for the algorithm, and show that its running time is given by the recurrence: $T(n) = 2T(n/2) + f(n)$ where $f(n) \in O(n \log n)$.
4. Show that the solution to this recurrence is $O(n \log^2 n)$.
5. Bring the running time down to $O(n \log n)$.

References and Exercises:

- Fast Selection [GT 9.2] [CLRS 9.3]

Exercise: Implement $\text{SPLIT}(S, x)$ in-place, i.e., by only swapping elements inside the array S .

- Black-box Minimization: https://en.wikipedia.org/wiki/Golden-section_search

Exercise: Implement it / write out the pseudocode

- Convex Hulls: with different algorithms [GT 22.2] [CLRS 33.3]

Exercise: Fill in the details of the other cases.

- Closest-Pair of Points: with different algorithms [22.4]

Exercise: Work through the analysis slide.

Next Week

More algorithmic techniques: Dynamic Programming

e.g., when greed is not enough